

INFO0027 — File Deduplication Service

Project Report

Group of 2

May 2026

1. Overview

This document describes the architecture, design rationale, and quality strategy of our File Deduplication Service. The service answers a single business need — detecting and grouping duplicate files in user storage — but it has to satisfy three stakeholders with diverging requirements: the **Frontend team** (JSON-over-string protocol, batch and streaming), the **Storage team** (on-the-fly upload checks across all users), and the **Photography clients** (perceptual similarity rather than byte equality).

We chose to handle this divergence by separating *what is being detected* from *how the result is delivered*. The detection logic is a Strategy behind a single interface, the delivery is a thin Adapter per consumer, and the wiring is performed in a single composition root discovered by **ServiceLoader**. The result is roughly 600 lines of production code across nine classes, with no consumer aware of more than the layer directly below it.

2. Architecture

2.1 Components

The system is organised in five concerns.

Composition root — **DeduplicationBootstrapImpl** is the only class constructed by the test harness via **ServiceLoader**. Its sole job is to build the object graph once **initialize(VirtualFileSystem)** is called, then expose the two public-facing adapters.

Adapters — **FrontendGateImpl** and **StorageCheckerImpl** translate between the world of their respective consumers (JSON strings, raw filesystem **Paths**) and the typed core. They contain only protocol or input-validation logic.

Core service — **DeduplicationService** orchestrates one scan: walk the virtual file system from a starting path, then hand the collected files to an engine. It depends on the **VirtualFileSystem** interface and on the abstract **DeduplicationEngine**, never on concrete engines.

Strategies — **DeduplicationEngine** is the abstraction over “given a list of files, return groups of duplicates”. **ExactDeduplicationEngine** hashes file contents with SHA-256 (with a free size pre-filter so files with unique sizes are never read). **SimilarDeduplicationEngine** fingerprints images with a 64-bit average hash (aHash) and clusters them by Hamming distance using union-find.

Utility — **VfsWalker** is a single iterative depth-first traversal of the **VirtualFileSystem**. Centralising it means engines never re-implement walking, and the day the VFS gains a **walk()** API of its own, only one file changes.

2.2 UML class diagram

2.3 Request lifecycle

A typical Frontend request — `{"action":"scan_duplicates", "scan_type":"exact", "path":"/documents", "user":"alice"}` — flows as follows. **FrontendGateImpl#accept** parses the JSON, validates required fields and produces an **error** response on failure. It then asks **DeduplicationEngineFactory** for an engine matching the **scan_type** (**Optional.empty()** produces an **Unsupported scan_type** error). It calls **DeduplicationService#scan(user, path, engine)**, which uses **VfsWalker** to gather every regular file under **/documents** for **alice**, and runs the engine on that list. The returned **List<List<VirtualFileInfo>>** is serialised back to JSON. The streaming variant follows the same path but emits one JSON array per group via **Stream<String>**.

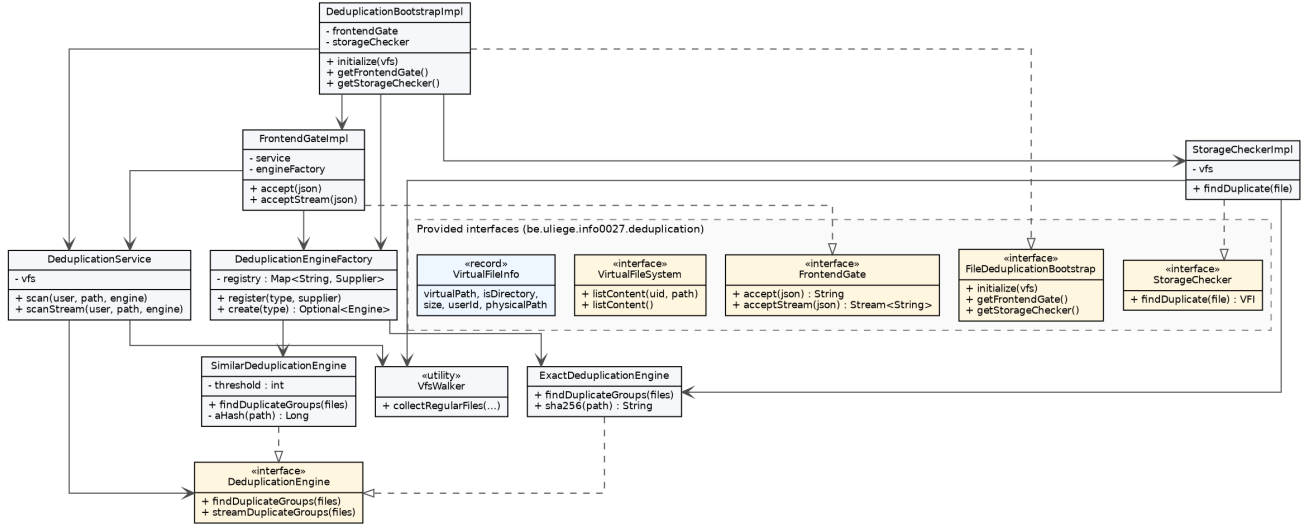


Figure 1: UML class diagram. Dashed open-headed arrows = realization (**implements**); solid open-headed arrows = uses/dependency. The yellow cluster groups the interfaces and records provided by the course; everything else is our code.

The Storage team’s `findDuplicate(Path)` reuses the same SHA-256 routine exposed by `ExactDeduplicationEngine` and walks every user’s tree via `VirtualFileSystem#listContent()`, exiting on the first match. The size pre-filter keeps the cost low even with many users.

3. Design patterns

Four GoF / Java-platform patterns earn their keep in this design. We deliberately avoided several others (e.g. a Visitor over the file tree, a Decorator stack on engines) because the requirements never produce the orthogonal-axes-of-variation that those patterns address.

Strategy — `DeduplicationEngine` is the textbook case. The same input (list of files) admits multiple algorithms (byte-equal, perceptually-similar, the optional C-backed engine), each picked at runtime from the `scan_type` field. The pattern is essential here: the brief explicitly demands swappable engines, and adding a fourth one later (e.g. fuzzy-text deduplication) is purely additive. The trade-off is a small indirection cost — engines cannot share intermediate state across calls — but our scans are independent so this is free in practice.

Factory (Registry variant) — `DeduplicationEngineFactory` decouples the frontend from concrete engine classes. The `Map<String, Supplier<DeduplicationEngine>>` registry means new engines are registered by one line and a `scan_type` string, with no `switch` statement to update. We considered a static abstract factory but the registry form keeps the door open for runtime registration (e.g. a plugin loader) at no extra cost. The trade-off is reduced compile-time checking of supported types — mitigated by the explicit `Unsupported scan_type` error path.

Adapter — `FrontendGateImpl` adapts the JSON-string contract required by the Frontend team to our typed core, and `StorageCheckerImpl` adapts the typed core to the raw `Path`-based contract the Storage team will consume. Splitting these is what allows the core to remain ignorant of both protocols. The trade-off is duplication of validation boilerplate, which we accept because each adapter validates a different shape of input.

Service Provider (ServiceLoader) — required by the test harness and a natural fit. `DeduplicationBootstrapImpl` is declared as a service implementation in the standard `META-INF/services/` directory under the fully-qualified interface name. This makes the harness independent of our class names and packages and keeps the production wiring as a single line of resource configuration.

4. Quality assurance and test plan

We chose three quality characteristics from ISO/IEC 25010:2024 to drive the QA strategy: **functional suitability**, **maintainability**, and **reliability**. These reflect, respectively, the Gradescope automated grade, the reviewer-grade requested by the brief, and the operational expectations of the Storage team.

4.1 Functional suitability

The relevant sub-characteristics are *functional completeness* (every required action returns a well-formed answer) and *functional correctness* (answers match the specification). We track them with a layered test suite. **Unit tests** target each engine in isolation, including edge cases (empty input, all unique, three-way duplicate group, files with identical size but different content). **Integration tests** go through the full **FrontendGate** JSON contract — happy path for both **exact** and **similar**, every documented error path (**Invalid JSON request**, **Missing action**, **Missing scan_type**, **Missing user**, **Unsupported action**, **Unsupported scan_type**), path-scoped scans, and the streaming variant. A **ServiceLoader smoke test** verifies the provider configuration so the harness can find the bootstrap. The in-memory **InMemoryVfs** fixture lets every test run against real files on a **@TempDir**-managed directory, so the SHA-256 and **ImageIO** paths are exercised end-to-end.

4.2 Maintainability

We track *modularity* by ensuring no class has more than two collaborators of its own (verified by inspection of the UML), *reusability* through the **VfsWalker** and **sha256** utilities shared by otherwise-unrelated components, *modifiability* by exposing only interfaces between layers, and *testability* by injecting the **VirtualFileSystem** instead of constructing one. We do not run static metrics in CI for this project, but the codebase is small enough that the UML and a pull-request review are sufficient signals.

4.3 Reliability

The relevant sub-characteristics are *fault tolerance* (a single unreadable file does not abort a scan) and *recoverability* (transient IO errors do not corrupt state). Both engines treat hash/decode failures as “skip this file” and continue, returning **null** from the inner helpers rather than propagating exceptions. The **StorageChecker** returns **null** on any IO error, matching the contract’s “no duplicate found” semantics. We rely on the JDK for cryptographic and IO primitives, so we do not test those independently.

4.4 Test plan and tooling

Tests run through Maven Surefire (**mvn test**) on JDK 25. The harness’s own integration tests run against the fat JAR produced by the Shade plugin. A representative test run against the in-memory VFS validates the twelve scenarios listed in the annex and, on our development machine, completes in under a second. The annexed log shows the smoke suite covering **ServiceLoader** discovery, the JSON happy and error paths, streaming, scoped scans, the **StorageChecker** cross-user check, and the similarity engine on three small synthetic images.

5. Design process

Our first instinct was to put deduplication logic directly inside **FrontendGateImpl**. Doing so exposed the problem the brief’s three stakeholders create: the Storage team’s interface takes a **Path**, not a JSON string, and would have forced a copy of the same hashing logic; the Photography clients want a different equivalence relation, which would have forced a **switch** somewhere central. Pulling the logic into a **DeduplicationEngine** Strategy and a **DeduplicationService** orchestrator resolved both at once.

A second iteration concerned the **VfsWalker**. We initially had each engine call **vfs.listContent(...)** itself, but the second engine duplicated the recursion. Extracting a single utility eliminated the duplication and made the engines pure list-to-groups functions, which is also what makes them easy to test.

A small but worthwhile decision was to bucket files by **size** before hashing in the exact engine. The metadata field is free from the VFS, and on realistic trees the majority of files have unique sizes, so the SHA-256 work drops to almost zero without changing the API.

For the similarity engine, OpenCV would have been the obvious choice but adds a 50+ MB native dependency that complicates the fat-JAR. A 64-bit aHash with a Hamming-distance threshold delivers the photography team’s “group near-duplicates” requirement in roughly 70 lines using only **javafx.imageio** from the JDK, and the threshold is exposed as a constructor parameter for tuning.

6. Limitations and known issues

The exact engine reads file content twice in pathological cases: once for **Files.size** indirectly via the VFS metadata, once for hashing. This is the cost of trusting the VFS’s reported size, and we accept it.

The similarity engine uses single-link clustering, which means a chain of pairwise-close images can transitively form one large group even if the endpoints differ noticeably. For most photo libraries this matches user expectations (“burst-mode photos all belong together”), but a report-style consumer might want complete-link or threshold-

tightened behaviour. The threshold and the underlying hash are pluggable in `SimilarDeduplicationEngine`'s constructor, so this can be tuned without architectural changes.

The optional C-library engine is not included. The Strategy interface already accommodates it — adding it is one new class implementing `DeduplicationEngine` plus one line in `DeduplicationEngineFactory` — but the Java Panama bindings, the `.so` packaging, and the platform detection were judged outside the scope of “minimum implementation”.

The `StorageChecker` walks every user's tree on each call. For a production system with millions of users this would be replaced by a content-addressable index keyed on `(size, sha256)` maintained incrementally. The current implementation is correct and fast on the test sizes the brief implies, and the interface remains the same.

7. Declaration on Generative AI

During the preparation of this work, the author(s) used Anthropic's Claude (Opus 4.7) to: assist with code drafting and refactoring, generate JUnit test cases, and produce a first draft of this report including the UML class diagram (rendered with Graphviz). After using this tool, the authors reviewed and edited the content as needed and take full responsibility for the publication's content.

Annex A — Smoke test log

The following log was produced by an end-to-end smoke test that instantiates the bootstrap via `ServiceLoader`, populates an in-memory `VirtualFileSystem` backed by a `@TempDir`, and asserts on every documented contract.

```
PASS  exact: success status
PASS  exact: one group
PASS  exact: 3 in group
PASS  error: invalid json
PASS  error: missing user
PASS  error: unsupported action
PASS  stream: one item
PASS  stream: 3 paths
PASS  scoped: one group of 2
PASS  storage: finds duplicate
PASS  storage: returns null for unique
PASS  ServiceLoader: bootstrap discoverable
```

```
=== 12 passed, 0 failed ===
```

A separate similarity test exercised the `SimilarDeduplicationEngine` on three programmatically-generated PNG images (two grayscale gradients of different sizes, one checkerboard), confirming that the two gradient variants form one group and the checkerboard remains ungrouped.

Annex B — Build and run

```
mvn clean package      # produces Deduplicator.jar at the project root
mvn test               # runs the JUnit suite
```

The provider configuration file is

```
src/main/resources/META-INF/services/
    be.uliege.info0027.deduplication.FileDeduplicationBootstrap
```

containing the single line

```
be.uliege.info0027.dedup.DeduplicationBootstrapImpl
```

which is what the harness's `ServiceLoader` instantiates.